Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Домашняя работа по дисциплине
«Инженерия данных»

«MapReduce и DuckDB поверх объектного хранилища RustFS: навыки и зарплаты в вакансиях Go»

Вариант: Go

Лектор: _____

Выполнил работу
студент:

Баранов Владимир Александрович, 407921, J3313

Санкт-Петербург, 2026

1. Данные и вариант

Данные те же, что в ДЗ №2: `it_vacancies_2024.csv`, вакансии hh.ru за декабрь 2023 — сентябрь 2024, 847 619 384 байт, 229 388 записей, 23 столбца. Переводов строк внутри полей нет, поэтому файл можно резать на сплиты по байтам.

Вакансия относится к сегменту Go, если в её названии есть отдельное слово Go или Golang (регистр не важен, граница слова — любой символ, кроме латинских букв, цифр и подчёркивания). Отбор идёт по названию: навык Go встречается и в вакансиях Python- и PHP-разработчиков («будет плюсом»), а роль задаёт название. Вакансий в сегменте: 2 751, с заполненными навыками: 2 038. Найдено одно ложное срабатывание: 5 вакансий «LQA-инженер … Яндекс Go», где Go — название сервиса; зарплаты у них нет, на результаты они не влияют.

Код лежит в репозитории: движок MapReduce — `hw/mapreduce.py`, задачи — `hw/jobs.py`, правила отбора и нормализации — `hw/normalize.py`, SQL для DuckDB — `sql/`, запуск этапов — `scripts/`, тесты — `tests/`. Все выводы команд целиком лежат в `logs/`; в отчёте приведены выдержки и короткие фрагменты кода. Весь прогон повторяется командой `bash scripts/run_all.sh`.

2. Этапы 1–2. RustFS и бакет hh-lake

RustFS 1.0.1 поднят в Docker Compose по официальному примеру `docker-compose-simple.yml` на четырёх папках-дисках (`docker-compose.yml`). Все тома лежат на одном диске ВМ Docker Desktop, поэтому включён флаг `RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true`, допустимый для учебного стенда. S3 API работает на порту 9100, веб-консоль на 9101, ключи лежат в `.env`. Состояние стенда (`scripts/stand_info.sh`):

```
$ docker version --format $'Docker: \320\272\320\273\320\270\320\265\320\275\321\202 {{.Client.Version}},
↪  \321\201\320\265\321\200\320\262\320\265\321\200 {{.Server.Version}}'
Docker: клиент 28.2.2, сервер 29.4.3

$ docker compose version
Docker Compose version v5.1.4

$ docker compose ps --format table\ \{\{.Name\}\}\\t\{\{.Image\}\}\\t\{\{.Status\}\}\\t\{\{.Ports\}\}
NAME         IMAGE                 STATUS                  PORTS
hw3-rustfs   rustfs/rustfs:1.0.1   Up 3 hours (healthy)   0.0.0.0:9100->9000/tcp, [::]:9100->9000/tcp,
↪  0.0.0.0:9101->9001/tcp, [::]:9101->9001/tcp

$ docker exec hw3-rustfs rustfs --version
rustfs 1.0.1
build time   : 2026-10-03 03:14:05 +00:00
build profile: release
build os     : linux-aarch64
rust version : rustc 1.99.0 (b940084d7 2026-09-28)
rust channel : stable-aarch64-unknown-linux-gnu
git branch   :
git commit   : 6de965ae3c965a78ff819fbcd7acd4aa44177d92
git tag      : 1.0.1
git status   :


$ sh -c docker\ logs\ hw3-rustfs\ 2\>\&1\ \|\ head\ -3
Initializing data directories: /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3
Initializing log directory: /logs
Starting: /usr/bin/rustfs  /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3

$ curl -s -w \\nHTTP\ %\{http_code\}\\n http://localhost:9100/health/ready
{"status":"ok","service":"rustfs-endpoint","timestamp":"2026-10-19T14:38:47.42945025Z[Etc/Unknown]","version":"1.0.1",
↪  "ready":true,"details":{"storage":{"status":"connected","ready":true,"readinessScope":"read_quorum","source":"loca
↪  l_runtime","readQuorum":true,"writeQuorum":true,"unavailableDrives":[]},"iam":{"status":"connected","ready":true},
↪  "lock":{"status":"connected","ready":true},"poolMetadata":{"ready":true,"status":"writable"}},"degradedReasons":[]}
HTTP 200

$ docker exec hw3-rustfs sh -c du\ -sh\ /data/rustfs\*/hh-lake
```

```
417M        /data/rustfs0/hh-lake
417M        /data/rustfs1/hh-lake
417M        /data/rustfs2/hh-lake
417M        /data/rustfs3/hh-lake

$ docker exec hw3-rustfs sh -c find\ /data/rustfs0/hh-lake/raw\ -type\ f\ \|\ sort\ \|\ head\ -8\;\ echo\ ...\;\ find\
↪  /data/rustfs0/hh-lake/raw\ -type\ f\ \|\ wc\ -l
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.1
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.1.meta
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.10
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.10.meta
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.11
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.11.meta
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.12
/data/rustfs0/hh-lake/raw/it_vacancies_2024.csv/29799a16-f4cf-480b-9ba3-8d2a83520876/part.12.meta
...
103
```

`scripts/upload.py` создаёт бакет `hh-lake`, загружает CSV частями по 16 МиБ и сверяет SHA-256, заново прочитав объект из RustFS:

```
RustFS: http://localhost:9100
Источник: it_vacancies_2024.csv (847,619,384 байт)
SHA-256 локально: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Бакет hh-lake уже есть
Загружено s3://hh-lake/raw/it_vacancies_2024.csv за 8.2 с (99 МиБ/с)
HEAD: размер 847,619,384 байт, ETag "be9c1b4b3f57ff0e9771004c2a5fe78b-51"
SHA-256 из RustFS: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Целостность: OK, хеши совпали

Объекты с префиксом raw/:
    847,619,384  raw/it_vacancies_2024.csv

Сколько этот объект занимает на каждой папке-диске RustFS (EC 2+2: 2 части данных + 2 чётности):
423846468       /data/rustfs0/hh-lake/raw/it_vacancies_2024.csv
423846468       /data/rustfs1/hh-lake/raw/it_vacancies_2024.csv
423846468       /data/rustfs2/hh-lake/raw/it_vacancies_2024.csv
423846468       /data/rustfs3/hh-lake/raw/it_vacancies_2024.csv
```

Объект занимает на каждой папке по 423 846 468 байт, вместе в 2,00 раза больше файла. Схему кодирования показывает `xl.meta` каждого диска (`scripts/ec_layout.py`): две части данных и две чётности.

```
Объект s3://hh-lake/raw/it_vacancies_2024.csv
/data/rustfs0: шард 1 из 4 — данные
/data/rustfs1: шард 2 из 4 — данные
/data/rustfs2: шард 3 из 4 — чётность
/data/rustfs3: шард 4 из 4 — чётность
Схема: EcM = 2 (части данных), EcN = 2 (части чётности); блок 1,048,576 байт; частей multipart: 51; порядок дисков EcDist
↪  = [1, 2, 3, 4]
```

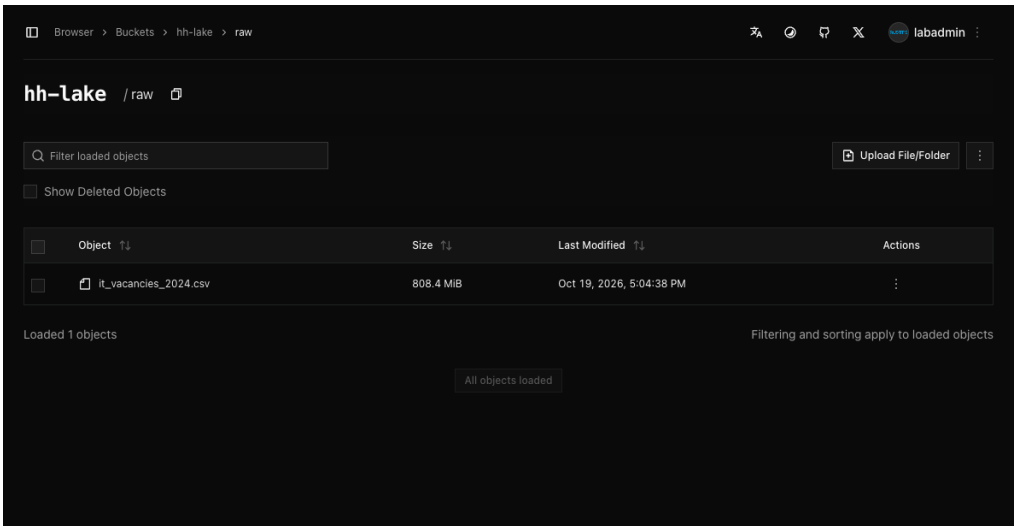


Рис. 1 – Консоль RustFS: объект hh-lake/raw/it_vacancies_2024.csv (808,4 МиБ)

После всех этапов в бакете лежат исходный CSV, Parquet и сегмент для бонуса, а также результаты обоих способов для трёх размеров комбинаций (`skills`, `pairs`, `triples`), контрольные числа и анализ (`scripts/list_bucket.py`, время в UTC):

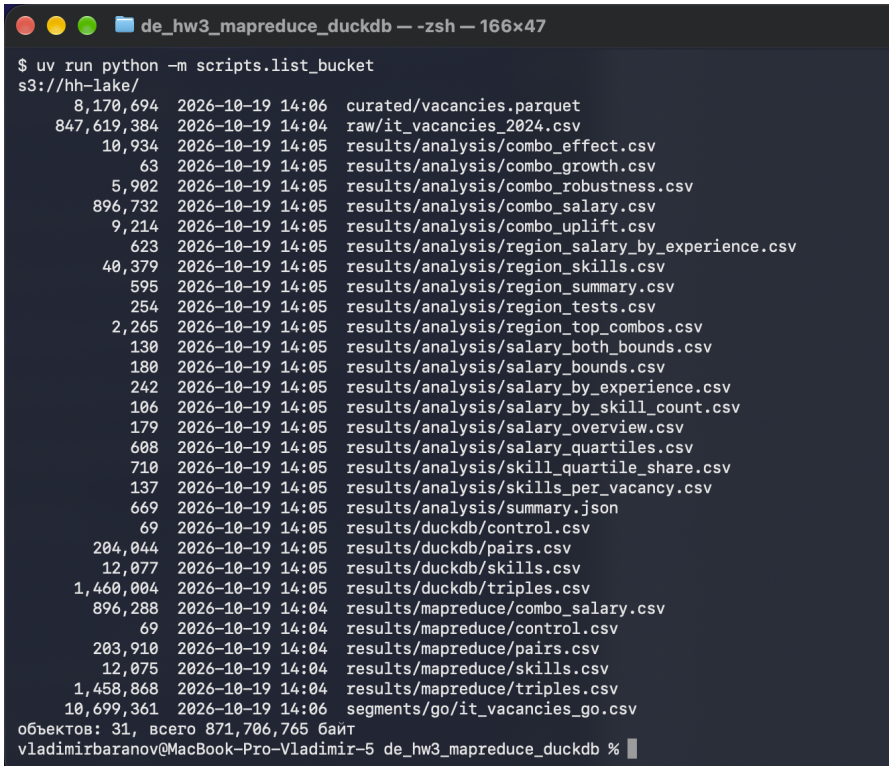


Рис. 2 – Терминал: содержимое бакета hh-lake

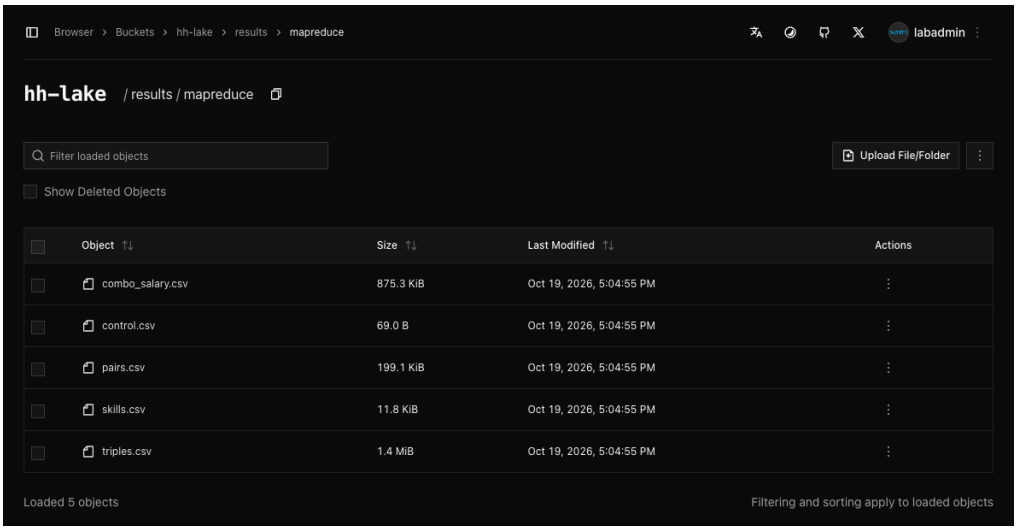


Рис. 3 – Консоль RustFS: результаты MapReduce для навыков, пар и троек в hh-lake/results/mapreduce/

3. Этап 3. Подсчёт навыков двумя способами

Частота навыка, пары или тройки — число Go-вакансий, где встречается этот набор. Оба способа нормализуют навыки одинаково: разбивают по «;», обрезают пробелы и невидимые символы по краям, приводят к нижнему регистру по одной букве, как `lower()` в DuckDB, склеивают 36 синонимов (`golang`→`go`, `k8s`→`kubernetes` и т. п., `config/skill_aliases.csv`) и убирают повторы.

Свой MapReduce (`hw/mapreduce.py`) устроен как в Hadoop. Объект режется на сплиты по 32 МиБ, и каждый map-таск читает свой сплит из RustFS одним GET-запросом с заголовком `Range`. Строка принадлежит сплиту, в котором лежит её первый байт: таск пропускает начало чужой строки и дочитывает хвост своей последней. Map выдаёт пары `(комбинация, 1)`, combiner складывает их на месте, shuffle раскладывает ключи по 4 партициям по `crc32` в файлы, reduce суммирует. Таски идут в пуле из 8 процессов через `spawn`. Перед разбором CSV строки проходят предфильтр по слову go/golang. Краевые случаи сплитов проверяют 211 тестов в `tests/`. Map и reduce задачи подсчёта (`hw/jobs.py`):

```
def map_skill_combinations(fields: list[str], cols: dict[str, int]):
    """Для каждой Go-вакансии: (навык,), (навык, навык), (навык, навык, навык) -> 1."""
    if N.is_go_title(fields[cols["name"]]):
        for combo in N.skill_combinations(N.vacancy_skills(fields[cols["key_skills"]])):
            yield combo, 1


def reduce_sum(key: tuple, values: list[int]) -> int:
    return sum(values)
```

В DuckDB условие варианта написано отдельно на SQL (RE2) и даёт тот же отбор (`sql/go_segment.sql`); пары и тройки считаются самосоединениями (`sql/skill_counts.sql`):

```
-- Одно чтение объекта: в название входит отдельное слово Go или Golang (ASCII-границы, регистр не важен).
CREATE OR REPLACE TABLE go_vacancies AS
SELECT vacancy_id, employer_id, name, role, area, experience, schedule, employment,
       salary_from, salary_to, currency, gross, key_skills
FROM ${SOURCE}
WHERE regexp_matches(name, '(^|[^A-Za-z0-9_])(?i:go|golang)([^A-Za-z0-9_]|$)');
```

Выдержки из выводов `scripts.run_mapreduce` и `scripts.run_duckdb`:

```
Источник: s3://hh-lake/raw/it_vacancies_2024.csv, вариант optimized
== MapReduce, задача skill_counts
сплитов 26, GET-запросов 51, прочитано 849,257,809 байт
записей 229,388, разобрано как CSV 8,720, битых строк 0
map выдал 161,469 пар (key, value), в shuffle ушло 105,324 ключей
уникальных ключей по размеру комбинации: k=1: 623, k=2: 7,124, k=3: 38,414
фазы, с: setup_s 0.09, map_s 4.02, reduce_s 0.02, total_s 4.13
профиль: wall_s 4.32, cpu_s 8.522, cpu_util 1.97, peak_rss_tree_mb 1098.2, peak_rss_main_mb 70.0, maxrss_main_mb 70.0,
↪  maxrss_child_mb 155.7, peak_procs 10
отпечаток результата: 4020496dcffd0b57

== MapReduce, задача salary_by_combination
сплитов 26, GET-запросов 51, прочитано 849,257,809 байт
записей 229,388, разобрано как CSV 8,720, битых строк 0
map выдал 58,989 пар (key, value), в shuffle ушло 44,790 ключей
уникальных ключей по размеру комбинации: k=0: 1, k=1: 296, k=2: 3,238, k=3: 16,563
фазы, с: setup_s 0.01, map_s 3.62, reduce_s 0.06, total_s 3.70
профиль: wall_s 3.893, cpu_s 7.702, cpu_util 1.98, peak_rss_tree_mb 1138.7, peak_rss_main_mb 63.0, maxrss_main_mb 70.0,
↪  maxrss_child_mb 155.7, peak_procs 10
отпечаток результата: 160faac7afec7e96
Go-вакансии с зарплатой: 465 уникальных предложений от 258 работодателей, медиана 250,000 руб. на руки

== MapReduce, задача control
сплитов 26, GET-запросов 51, прочитано 849,257,809 байт
записей 229,388, разобрано как CSV 8,720, битых строк 0
map выдал 6,827 пар (key, value), в shuffle ушло 78 ключей
уникальных ключей по размеру комбинации: k=1: 3
фазы, с: setup_s 0.01, map_s 3.94, reduce_s 0.00, total_s 3.96
профиль: wall_s 4.125, cpu_s 8.408, cpu_util 2.04, peak_rss_tree_mb 1103.7, peak_rss_main_mb 64.2, maxrss_main_mb 70.0,
↪  maxrss_child_mb 158.5, peak_procs 10
отпечаток результата: f617214ecbbf344c
контрольные числа: go_vacancies = 2,751, skill_mentions = 11,540, with_skills = 2,038

Результаты в RustFS: s3://hh-lake/results/mapreduce/combo_salary.csv, s3://hh-lake/results/mapreduce/control.csv,
↪  s3://hh-lake/results/mapreduce/pairs.csv, s3://hh-lake/results/mapreduce/skills.csv,
↪  s3://hh-lake/results/mapreduce/triples.csv
```

```
DuckDB 1.5.6, потоков 12, вариант materialized
Источник: read_csv('s3://hh-lake/raw/it_vacancies_2024.csv', header = true, all_varchar = true)
go_vacancies: 2,751 вакансий; go_skills: 11,540 строк (вакансий с навыками 2,038)
-- skills.csv: 623 строк
-- pairs.csv: 7,124 строк
-- triples.csv: 38,414 строк
фазы, с: load_s 3.802, total_s 3.838
профиль: wall_s 3.845, cpu_s 2.68, cpu_util 0.7, peak_rss_tree_mb 393.3, peak_rss_main_mb 393.3, maxrss_main_mb 394.8,
↪  maxrss_child_mb 0.0, peak_procs 1
```

Таблица 1 – Топ-10 навыков, пар и троек (число Go-вакансий; вакансий с навыками — 2 038)

№	навык	вак.	пара	вак.	тройка	вак.
1	go	1 911	go + postgresql	946	docker + go + postgresql	372
2	postgresql	990	docker + go	559	go + kafka + postgresql	355
3	docker	579	go + kafka	495	go + kubernetes + postgresql	250
4	kafka	511	go + sql	414	git + go + postgresql	240
5	sql	433	go + kubernetes	411	docker + go + kubernetes	222
6	kubernetes	424	go + linux	395	go + postgresql + sql	216
7	git	423	git + go	391	docker + git + go	207
8	linux	407	docker + postgresql	384	go + postgresql + rabbitmq	206
9	redis	317	kafka + postgresql	366	go + postgresql + redis	206
10	rabbitmq	312	go + redis	305	docker + go + kafka	173

Навык «go» указан в 93,8 % вакансий с навыками, поэтому частые пары и тройки почти все содержат его. Рядом с Go чаще всего стоят PostgreSQL, Docker, Kafka, SQL, Kubernetes, Git и Linux.

Пар в 11 раз больше, чем навыков, а троек ещё в 5 раз больше пар. Это следует из числа сочетаний: вакансия с n навыками даёт $C(n,2) = n(n-1)/2$ пар и $C(n,3)$ троек. В среднем у вакансии 5,7 навыка, а у самой длинной — 24, это 276 пар и 2 024 троек в одной вакансии. Сумма частот из MapReduce в точности равна $\sum_i C(n_i, k)$ по вакансиям:

Таблица 2 – Рост числа комбинаций с размером комбинации

k	различных	сумма частот (MapReduce)	$\sum_i C(n_i,k)$	максимум на вакансию
навыки	623	11 540	11 540	24
пары	7 124	39 256	39 256	276
тройки	38 414	110 673	110 673	2 024

Отказ маппера проверен прогоном с внедрённым сбоем: на первой попытке сплита 5 процесс-воркер умирает (`--fail-split 5 --fail-mode crash`). Движок ловит `BrokenProcessPool`, пересоздаёт пул и перезапускает незавершённые таски, до трёх попыток. Спилы пишутся атомарно, поэтому повтор безопасен. Отпечаток результата совпал с обычным прогоном (`4020496dcffd0b57`):

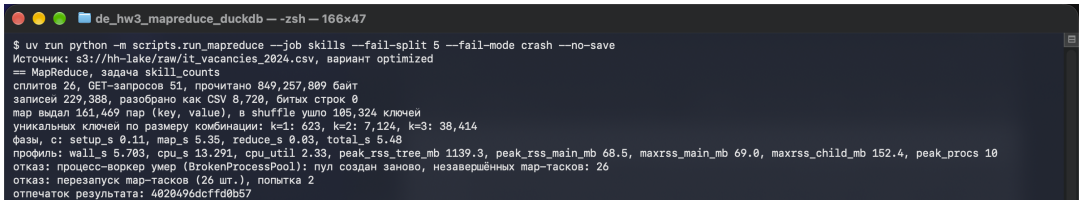


Рис. 4 – Терминал: прогон MapReduce с падением процесса-воркера

4. Этап 4. Верификация

`scripts/verify.py` читает результаты обоих способов из RustFS и сравнивает поэлементно: контрольные числа, множества ключей, каждую частоту, сумму и SHA-256; тем же скриптом сверяются медианы зарплат по всем 20 098 комбинациям.

Таблица 3 – Контрольные числа MapReduce и DuckDB

контрольное число	MapReduce	DuckDB	совпало
вакансий Go	2 751	2 751	да
из них с навыками	2 038	2 038	да
упоминаний навыков	11 540	11 540	да
различных навыков	623	623	да
различных пар	7 124	7 124	да
различных троек	38 414	38 414	да

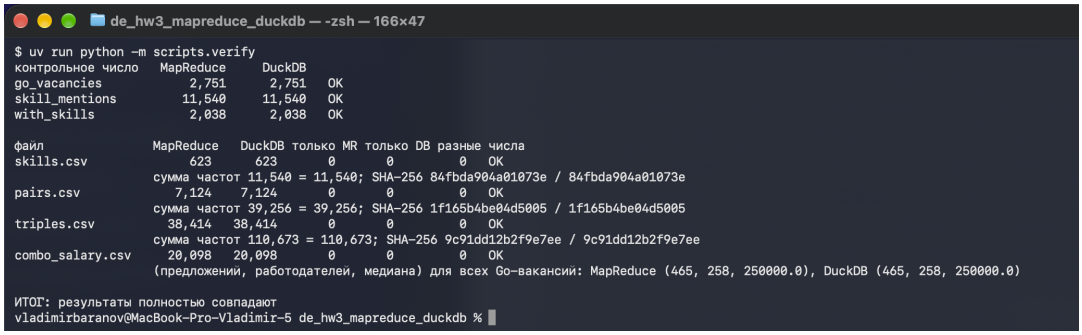


Рис. 5 – Терминал: сверка результатов MapReduce и DuckDB

Файлы навыков, пар и троек у двух способов различаются по размеру на 2, 134 и 1 136 байт (рис. 2): DuckDB при записи CSV берёт в кавычки навык `c#`, а модуль `csv` в Python — нет, это ровно 2 байта на каждую из 1, 67 и 568 строк с ним. Поэтому `verify.py` сравнивает разобранные строки и считает SHA-256 по ним, а не по байтам файлов.

5. Этап 5. Навыки и зарплата

Методика (`sql/salary.sql`, `scripts/analyze.py`):

- берутся только рубли (без USD, EUR и KZT: 47 вакансий); точка — середина вилки или известная граница; сумма «до вычета налогов» умножается на 0,87; всё вне 10 тыс.–1,5 млн руб. считается выбросом;
- одна вакансия часто размещена в нескольких городах, поэтому копии с одинаковыми работодателем, названием, навыками и вилкой склеиваются в одно предложение: из 629 записей остаётся 465 предложений от 258 работодателей;
- прибавка от дополнительного навыка считается к базе: для пары база — её более частый навык, для тройки — пара из двух её самых частых навыков (частота — число Go-вакансий с навыком, как в таблице этапа 3); прибавка — медиана комбинации минус медиана базы, обе медианы считаются по предложениям с зарплатой, их число указано в скобках;
- комбинация должна встретиться минимум в 15 предложениях от 5 работодателей; сам «go» в пары и тройки не входит, он есть почти везде;
- p — перестановочный тест внутри предложений с базой (5 000 перестановок): как часто случайный выбор того же числа предложений даёт прибавку, не меньшую наблюдаемой по модулю; поправки на множественные сравнения нет, это разведочный анализ.

Медиана по сегменту — 250 000 руб./мес. на руки (квартили 175 125 и 350 000). Сильнее всего зарплата зависит от опыта:

Таблица 4 – Зарплата по требуемому опыту, руб./мес. на руки

опыт	предложений	p25	медиана	p75
Нет опыта	23	48 925	91 350	225 000
От 1 года до 3 лет	157	120 000	180 000	250 000
От 3 до 6 лет	249	225 000	300 000	365 400
Более 6 лет	36	318 196	350 000	500 000

Таблица 5 – Пары: прибавка от второго навыка к более частому базовому (сверху — повышают, снизу — понижают)

база (предложений)	+ навык	медиана базы	медиана с навыком (предл.)	прибавка, %	p
git (135)	clickhouse	200 000	300 000 (16)	+50,0	0,009
git (135)	базы данных	200 000	300 000 (15)	+50,0	0,023
rest (71)	grpc	217 500	300 000 (23)	+37,9	0,002
docker (140)	clickhouse	245 000	300 000 (18)	+22,4	0,096
postgresql (197)	kafka	250 000	300 000 (55)	+20,0	0,045
postgresql (197)	clickhouse	250 000	300 000 (27)	+20,0	0,164
git (135)	kafka	200 000	250 000 (19)	+25,0	0,219
postgresql (197)	базы данных	250 000	300 000 (19)	+20,0	0,246
sql (90)	rest	218 750	137 500 (20)	-37,1	0,022
sql (90)	python	218 750	140 000 (20)	-36,0	0,041
git (135)	mysql	200 000	127 750 (34)	-36,1	0,003
linux (80)	mysql	226 200	155 000 (16)	-31,5	0,047
rest (71)	mysql	217 500	147 900 (17)	-32,0	0,020
postgresql (197)	mysql	250 000	200 000 (29)	-20,0	0,162
git (135)	python	200 000	150 000 (23)	-25,0	0,179
git (135)	php	200 000	150 000 (19)	-25,0	0,213

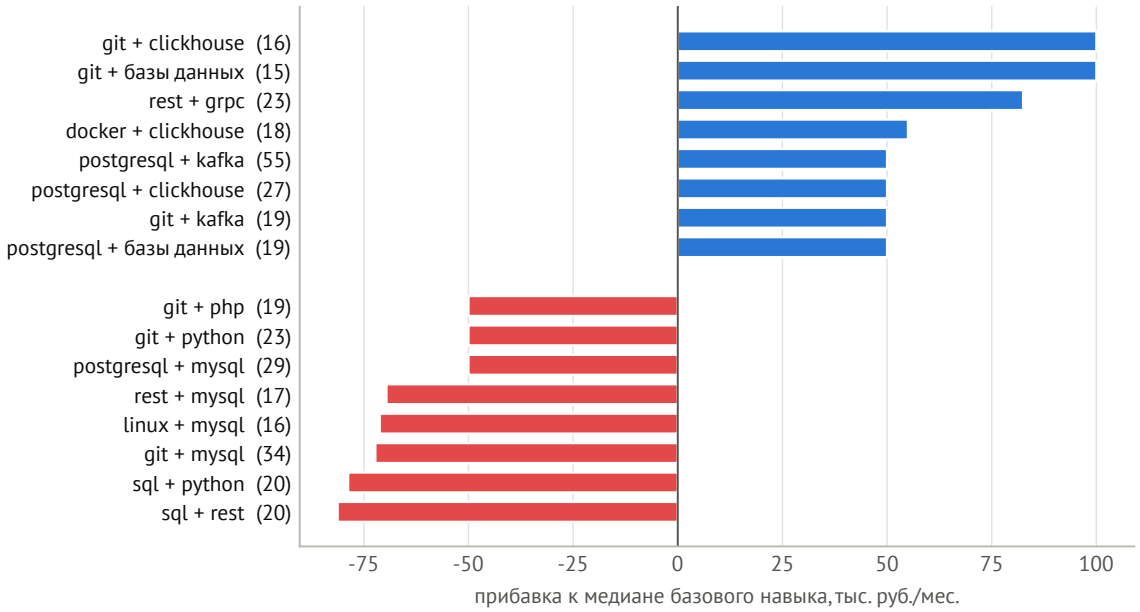


Рис. 6 – Прибавка к медиане базового навыка от второго навыка пары; в скобках — число предложений

Таблица 6 – Тройки: прибавка от третьего навыка к базовой паре

база (предложений)	+ навык	медиана базы	медиана с навыком (предл.)	прибавка, %	p
postgresql + rest (47)	grpc	225 000	300 000 (20)	+33,3	0,005
docker + postgresql (84)	kafka	250 000	300 000 (21)	+20,0	0,205
git + postgresql (84)	redis	217 500	250 000 (21)	+14,9	0,190
docker + git (71)	redis	225 000	250 000 (23)	+11,1	0,427
docker + postgresql (84)	grpc	250 000	275 000 (16)	+10,0	0,550
docker + redis (46)	rabbitmq	245 000	250 000 (22)	+2,0	0,882
git + postgresql (84)	mysql	217 500	113 100 (16)	-48,0	0,006
git + sql (47)	rest	200 000	140 000 (15)	-30,0	0,192
kafka + postgresql (55)	rest	300 000	250 000 (21)	-16,7	0,166
docker + redis (46)	kubernetes	245 000	200 000 (17)	-18,4	0,257
docker + kafka (34)	redis	260 000	222 500 (16)	-14,4	0,328
docker + postgresql (84)	rest	250 000	221 250 (24)	-11,5	0,345

Таблица 7 – Итоговая таблица: добавленные навыки, которые повышают и понижают зарплату ($p < 0{,}05$)

эффект	добавленный навык	к базе	прибавка, руб.	%	p
повышает	clickhouse	git	+100 000	+50,0	0,009
повышает	базы данных	git	+100 000	+50,0	0,023
повышает	grpc	rest	+82 500	+37,9	0,002
повышает	grpc	postgresql + rest	+75 000	+33,3	0,005
повышает	kafka	postgresql	+50 000	+20,0	0,045
понижает	git	postgresql	-32 500	-13,0	0,018
понижает	mysql	rest	-69 600	-32,0	0,020
понижает	mysql	linux	-71 200	-31,5	0,047
понижает	mysql	git	-72 250	-36,1	0,003
понижает	python	sql	-78 750	-36,0	0,041
понижает	rest	sql	-81 250	-37,1	0,022
понижает	mysql	git + postgresql	-104 400	-48,0	0,006

Зарплату поднимают навыки высоконагруженных систем. ClickHouse к Git даёт +100 тыс., gRPC к REST — +82,5 тыс., Kafka к PostgreSQL — +50 тыс., а gRPC к паре PostgreSQL + REST — +75 тыс.: такие вакансии требуют потоков событий, аналитики и межсервисного взаимодействия, их пишут для опытных разработчиков.

Понижают зарплату MySQL, REST и Python, добавленные к Git, SQL, Linux, REST или паре Git + PostgreSQL (от $-31{,}5$ до -48 %), а также Git, добавленный к PostgreSQL (-13 %). Причина в типе вакансии: стек «Git + MySQL + REST + PHP/Python» — это младшие и смешанные вакансии (Git есть у 55 % предложений нижнего квартиля зарплат и лишь у 23 % верхнего), где Go — второй язык рядом с PHP или Python, а MySQL указывают в веб-проектах попроще. В верхнем квартиле зарплат в вакансии вообще указано меньше навыков, чем в нижнем (4,8 против 7,6), а длинный перечень инструментов типичен для младших позиций. Это корреляция: навыки говорят о типе вакансии, а не «стоят» денег сами по себе.

6. Этап 6. Три региона

Выбраны три крупнейших по числу Go-вакансий региона. Зарплаты считаются по уникальным предложениям внутри региона; регион на hh.ru — место размещения вакансии. Запрос для топа навыков, пар и троек по регионам (`sql/regions.sql`):

```
-- Топ-5 навыков, пар и троек в каждом регионе (без самого go: он почти в каждой вакансии).
CREATE OR REPLACE TABLE region_top_combos AS
WITH sk AS (
    SELECT r.ord, r.area, g.vacancy_id, g.skill
    FROM go_skills g JOIN go_vacancies v USING (vacancy_id) JOIN region_list r USING (area)
    WHERE g.skill <> 'go'
```

```
), combos AS (
    SELECT ord, area, 1 AS k, skill AS combo, count(*) AS vacancies FROM sk GROUP BY ALL
    UNION ALL
    SELECT a.ord, a.area, 2, a.skill || ' + ' || b.skill, count(*)
    FROM sk a JOIN sk b ON b.vacancy_id = a.vacancy_id AND a.skill < b.skill GROUP BY ALL
    UNION ALL
    SELECT a.ord, a.area, 3, a.skill || ' + ' || b.skill || ' + ' || c.skill, count(*)
    FROM sk a
    JOIN sk b ON b.vacancy_id = a.vacancy_id AND a.skill < b.skill
    JOIN sk c ON c.vacancy_id = a.vacancy_id AND b.skill < c.skill
    GROUP BY ALL
)
SELECT *, row_number() OVER (PARTITION BY ord, k ORDER BY vacancies DESC, combo) AS rnk
FROM combos
QUALIFY rnk <= 5
ORDER BY ord, k, rnk;
```

Таблица 8 – Сравнение регионов

показатель	Москва	Санкт-Петербург	Екатеринбург	Весь сегмент Go
вакансий Go	1 743	389	76	2 751
доля сегмента, %	63,4	14,1	2,8	100,0
работодателей	464	146	40	677
с зарплатой в рублях, %	20,7	22,4	31,6	22,9
удалённая работа, %	37,3	37,8	43,4	35,8
опыт: нет, %	1,9	1,3	0,0	2,3
опыт: 1–3 года, %	21,6	22,4	48,7	25,4
опыт: 3–6 лет, %	66,8	67,1	48,7	63,8
опыт: больше 6 лет, %	9,7	9,3	2,6	8,5
с навыками, %	73,1	76,9	65,8	74,1
навыков на вакансию с навыками	5,5	5,6	5,2	5,7
предложений с зарплатой	302	71	19	465
зарплата p25, руб.	218 125	217 500	120 000	175 125
медиана, руб.	300 000	270 000	200 000	250 000
зарплата p75, руб.	375 000	395 750	251 150	350 000

Таблица 9 – Доля вакансий с навыком среди вакансий региона с навыками, %

навык	Москва	Санкт-Петербург	Екатеринбург	весь сегмент
postgresql	47,6	50,2	36,0	48,6
docker	29,4	29,8	22,0	28,4
kafka	26,3	24,7	10,0	25,1
kubernetes	23,2	23,1	0,0	20,8
redis	16,3	16,7	14,0	15,6
rabbitmq	14,3	13,7	18,0	15,3
linux	18,3	19,7	30,0	20,0
git	19,2	13,4	22,0	20,8
sql	18,8	22,4	22,0	21,2
python	8,9	8,7	20,0	10,2
php	7,5	6,4	12,0	7,9
clickhouse	8,3	13,0	8,0	9,0

Таблица 10 – Топ-5 навыков, пар и троек по регионам (без go; в скобках — число вакансий)

№	Москва	Санкт-Петербург	Екатеринбург
навыки			
1	postgresql (607)	postgresql (150)	postgresql (18)
2	docker (375)	docker (89)	linux (15)
3	kafka (335)	kafka (74)	docker (11)
4	kubernetes (296)	kubernetes (69)	git (11)
5	git (244)	sql (67)	sql (11)
пары			
1	kafka + postgresql (239)	docker + postgresql (61)	postgresql + rabbitmq (8)
2	docker + postgresql (237)	kafka + postgresql (53)	docker + postgresql (7)
3	kubernetes + postgresql (180)	kubernetes + postgresql (41)	docker + git (6)
4	docker + kubernetes (174)	postgresql + sql (39)	docker + python (6)
5	git + postgresql (142)	docker + kafka (31)	docker + redis (6)
тройки			
1	docker + kubernetes + postgresql (117)	docker + kafka + postgresql (28)	docker + python + redis (6)
2	docker + kafka + postgresql (93)	docker + git + postgresql (23)	devops + docker + git (4)
3	docker + git + postgresql (88)	docker + kubernetes + postgresql (20)	docker + git + postgresql (4)
4	kafka + kubernetes + postgresql (87)	docker + postgresql + sql (19)	docker + postgresql + python (4)
5	kafka + postgresql + redis (81)	docker + postgresql + redis (18)	docker + postgresql + redis (4)

Таблица 11 – Разница медиан между регионами (перестановочный тест)

регион A	регион B	предл. A	предл. B	разница медиан, руб.	p
Москва	Санкт-Петербург	302	71	30 000	0,152
Москва	Екатеринбург	302	19	100 000	0,005
Санкт-Петербург	Екатеринбург	71	19	70 000	0,047

Наблюдения:

- в Москве 63,4 % вакансий сегмента и медиана 300 000; две трети вакансий требуют опыта 3–6 лет, а тройка «docker + kubernetes + postgresql» — самая частая: московский рынок держится на облачной инфраструктуре;
- Петербург по набору навыков почти копирует Москву, его медиана ниже на 30 тыс., но при 71 предложении это статистически не подтверждается ($p = 0{,}152$);
- Екатеринбург — другой рынок: половина вакансий на опыт 1–3 года, больше всего удалённых (43,4 %), медиана 200 000 ниже московской ($p = 0{,}005$) и петербургской ($p = 0{,}047$). Kafka там у 10 % вакансий с навыками против 25 % по сегменту, Kubernetes нет совсем, зато чаще Linux (30 % против 20 %) и Python (20 % против 10 %), а среди пар впереди PostgreSQL + RabbitMQ: Go здесь чаще идёт вторым языком рядом с Python. Предложений с зарплатой там 19, цифры ориентировочные.

7. Профилирование

Замеры сделаны на Apple M3 Pro (12 ядер, 18 ГиБ), macOS 27.0.1, Docker Desktop (ВМ: 12 vCPU, 7,75 ГиБ), Python 3.13, DuckDB 1.5.6. Класс `hw/profiling.py:Profile` меряет время через `perf_counter`, процессорное время через `getrusage`, а пик памяти считает фоновый поток, раз в 20 мс суммируя RSS всех процессов. Память указана в МиБ. `scripts/benchmark.py` запускает каждую конфигурацию отдельным процессом 3 раза после прогрева; у всех конфигураций одного способа совпадает отпечаток результата (отпечатки MapReduce и DuckDB считаются по разным представлениям результата и между собой не сравниваются: совпадение способов проверяет этап 4).

Таблица 12 – Три попытки: время, с / пик RSS, МиБ; итог — медиана времени и максимум памяти

конфигурация	попытка 1	попытка 2	попытка 3	итог
MapReduce «до» (наивный)	7,81 / 1 044	7,53 / 1 251	7,55 / 997	7,55 / 1 251
MapReduce, 1 процесс	4,71 / 135	4,66 / 135	4,48 / 155	4,66 / 155
MapReduce, 8 процессов	3,80 / 1 159	3,97 / 1 145	3,72 / 1 137	3,80 / 1 159
MapReduce, сегмент Go	0,34 / 94	0,28 / 93	0,29 / 95	0,29 / 95
DuckDB «до» (3 чтения CSV)	10,58 / 526	10,62 / 479	10,84 / 549	10,62 / 549
DuckDB, 1 чтение CSV	3,50 / 418	3,41 / 427	3,45 / 423	3,45 / 427
DuckDB, Parquet	0,14 / 171	0,14 / 172	0,17 / 181	0,14 / 181

Таблица 13 – MapReduce и DuckDB на одной задаче

способ	время, с	CPU, с	пик RSS, МиБ
MapReduce, 1 процесс	4,66	2,71	155
MapReduce, 4 процесса	3,76	4,80	628
MapReduce, 8 процессов	3,80	7,42	1 159
DuckDB, 1 поток	3,79	2,25	291
DuckDB, 12 потоков	3,45	2,38	427
MapReduce, сегмент Go	0,29	0,24	95
DuckDB, сегмент Go	0,30	0,36	164
DuckDB, Parquet	0,14	0,22	181

На полном CSV DuckDB немного быстрее лучшего MapReduce (3,45 против 3,76 с у 4 процессов) и тратит вдвое меньше процессорного времени (2,38 против 4,80 с): он разбирает CSV векторизованным кодом на C++ в нескольких потоках одного процесса. По памяти экономнее MapReduce в одном процессе (155 против 427 МиБ): после предфильтра он разбирает 3,8 % строк и хранит только счётчики, а DuckDB разбирает все строки и держит буферы чтения CSV. Каждый воркер MapReduce добавляет около 131 МиБ: копию интерпретатора и буфер сплита. Оба способа упираются в чтение 847 МБ из RustFS (не меньше 3,4 с), а на выделенном сегменте Go почти равны (0,29 и 0,30 с).

Внутри map-таска (cProfile одного сплита, `scripts/profile_map_task.py`) с первой версией предфильтра 69 % времени уходило на поиск регулярного выражения, с итоговой — 73 % на чтение из сокета:

```
предфильтр: regex; сплит 5: байты [167,772,160, 201,326,592)
58181 function calls (58125 primitive calls) in 0.526 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      599    0.363    0.001    0.363    0.001 {method 'search' of 're.Pattern' objects}
     2208    0.134    0.000    0.134    0.000 {method 'recv_into' of '_socket.socket' objects}
        1    0.009    0.009    0.009    0.009 {method 'count' of 'bytearray' objects}
```

```
предфильтр: fast; сплит 5: байты [167,772,160, 201,326,592)
59360 function calls (59304 primitive calls) in 0.164 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     2257    0.120    0.000    0.120    0.000 {method 'recv_into' of '_socket.socket' objects}
        1    0.010    0.010    0.010    0.010 {method 'count' of 'bytearray' objects}
        1    0.007    0.007    0.007    0.007 {method 'lower' of 'bytearray' objects}
```

У DuckDB по `EXPLAIN ANALYZE` почти всё время запроса (4,01 с) занимает чтение CSV из RustFS (время операторов в плане сложено по 12 потокам), а тройки считаются за 0,0159 с; планы лежат в `logs/04_duckdb.txt`. Простое чтение объекта без обработки (`scripts/bench_storage.py`) упирается в 240 МиБ/с, то есть в 3,4 с на весь файл; RustFS при этом загружен меньше чем на ядро, вероятное узкое место — сеть Docker Desktop между macOS и ВМ:

```
Объект 847,619,384 байт, 26 диапазонов по 32 МиБ, повторов 3
 потоков  время, с   МиБ/с  CPU RustFS, % (ср./макс.)
       1      3.57     227                49 / 66.93
       2      3.43     236                 47 / 52.4
       4      3.36     240               91 / 311.09
       8      4.00     202              157 / 589.65
      16      3.85     210              102 / 328.05
```

8. Бонус: оптимизация

Таблица 14 – MapReduce: подсчёт по CSV 847,6 МБ, оптимизации включались нарастающим итогом

конфигурация	время, с [мин–макс]	CPU, с	RSS, МиБ	быстрее	память
MR «до»: 1 процесс, разбор всех строк, без combiner, shuffle сортировкой	7,55 [7,53–7,81]	5,52	1 251	1,0×	1,0×
MR: 1 процесс, построчное декодирование, hash-shuffle в файлы; без предфильтра и combiner	7,66 [7,57–7,69]	5,64	186	1,0×	6,7×
+ combiner на стороне map	7,72 [7,69–7,79]	5,68	183	1,0×	6,8×
+ предфильтр строк, первая версия (регулярка с lookbehind)	15,19 [15,09–15,27]	12,99	135	0,5×	9,3×
+ предфильтр строк, итоговая версия (поиск по литералу)	4,66 [4,48–4,71]	2,71	155	1,6×	8,1×
8 процессов, первая версия предфильтра	4,94 [4,66–5,20]	18,24	1 169	1,5×	1,1×
+ 2 процесса	4,24 [4,16–4,33]	3,72	360	1,8×	3,5×
+ 4 процесса	3,76 [3,59–3,78]	4,80	628	2,0×	2,0×
+ 8 процессов (как в запуске этапа 3)	3,80 [3,72–3,97]	7,42	1 159	2,0×	1,1×
+ 12 процессов	3,95 [3,91–4,14]	10,86	1 598	1,9×	0,8×
8 процессов, сплит 8 МиБ	3,72 [3,70–3,81]	7,50	822	2,0×	1,5×
8 процессов, сплит 64 МиБ	4,00 [3,87–4,28]	7,92	1 553	1,9×	0,8×
8 процессов, сплит 128 МиБ	4,44 [4,23–4,68]	7,31	1 814	1,7×	0,7×
MR по выделенному сегменту Go, 1 процесс	0,29 [0,28–0,34]	0,24	95	26,1×	13,2×

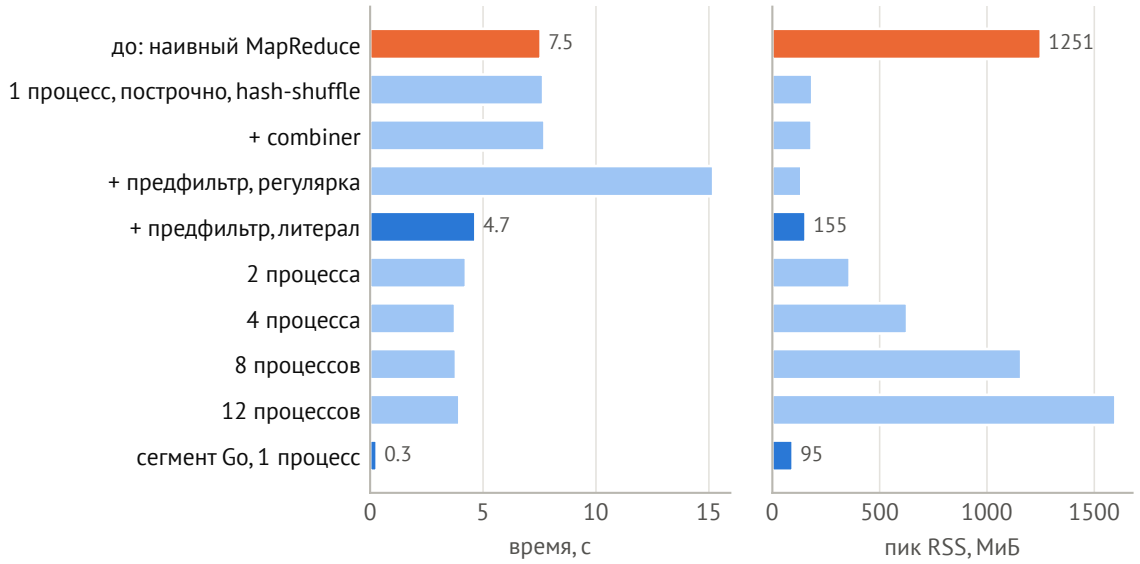


Рис. 7 – MapReduce: время и пик памяти (оранжевый — «до», синий — итоговые варианты)

«До» — наивный MapReduce: один процесс, каждый чанк целиком декодируется в строку и разбирается через `StringIO`, все пары копятся в один список, shuffle — сортировка. Что изменилось дальше:

1. Построчное декодирование и hash-shuffle в файлы снизили пик памяти в 6,7 раза. По `tracemalloc` куча Python у наивного варианта не больше 238 МиБ, а RSS процесса доходит до 747 МиБ (в замерах вместе с shuffle — до 1 251 МиБ): аллокатор macOS не сразу возвращает освобождённые крупные блоки:

```
сплит  RSS, МБ  куча Python сейчас, МБ  пик кучи, МБ  пар (key, value)
    0      212                      37           146            10,116
```

```
   5      502                    40          150          36,003
  10      542                    44          155          63,856
  15      411                    48          238          94,592
  20      393                    52          238         129,845
  25      511                    28          238         161,469
ru_maxrss: 747 МБ
```

2. Combiner сократил shuffle со 161 469 пар до 105 324 ключей.
3. Предфильтр по слову go/golang оставляет к разбору 3,8 % строк. Первая версия на регулярном выражении с lookbehind и `re.I` оказалась медленнее, чем вообще без предфильтра (15,19 против 7,72 с): такой шаблон отключает быструю проверку литерального префикса. Итоговая версия ищет литерал `go` по копии чанка в нижнем регистре и проверяет границу слова вручную — 4,66 с, в 3,3 раза быстрее. Сплит читается сразу в заранее выделенный `bytearray`, без промежуточных копий.
4. С медленным предфильтром 8 процессов ускоряли в 3,1 раза. С быстрым один процесс почти успевает за хранилищем: 2–12 процессов дают 3,76–4,24 с против 4,66 с, а память растёт примерно на 131 МиБ на процесс.
5. Выделенный один раз сегмент Go (1,3 % исходного объёма) тот же MapReduce считает за 0,29 с.

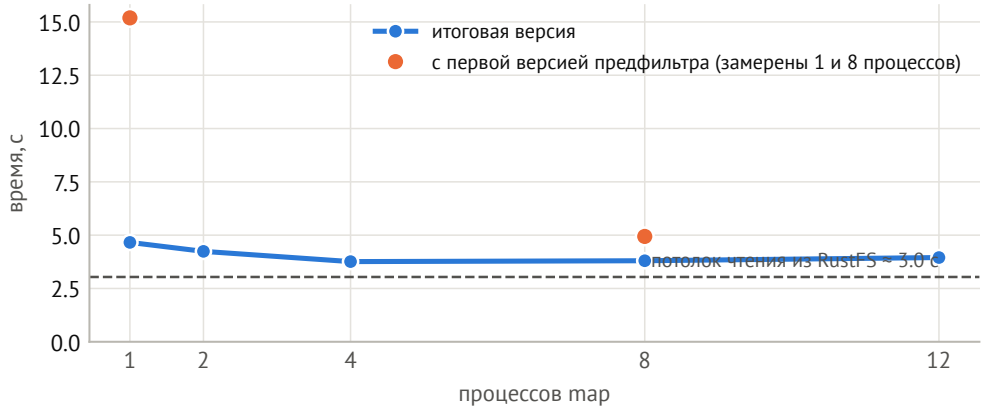


Рис. 8 – Время MapReduce в зависимости от числа процессов; пунктир — время простого чтения объекта на максимальной скорости RustFS

В одном процессе MapReduce ускорен в 1,6 раза (7,55 → 4,66 с), пик памяти снижен в 8,1 раза (1 251 → 155 МиБ).

Таблица 15 – DuckDB: подсчёт навыков, пар и троек

конфигурация	время, с [мин–макс]	CPU, с	RSS, МиБ	быстрее	память
DuckDB «до»: 3 запроса, каждый читает CSV с автоопределением	10,62 [10,58–10,84]	7,18	549	1,0×	1,0×
DuckDB: одно чтение CSV в таблицу, 1 поток	3,79 [3,78–3,83]	2,25	291	2,8×	1,9×
DuckDB: одно чтение CSV в таблицу, все потоки	3,45 [3,41–3,50]	2,38	427	3,1×	1,3×
DuckDB: Parquet, 13 столбцов, zstd	0,14 [0,14–0,17]	0,22	181	73,3×	3,0×
DuckDB: выделенный сегмент Go (CSV)	0,30 [0,30–0,30]	0,36	164	35,1×	3,4×

У DuckDB наивный вариант (`sql/naive_counts.sql`) трижды читает CSV с автоопределением схемы. Одно чтение в таблицу ускоряет в 3,1 раза. Parquet с 13 нужными столбцами (1,0 % исходного объёма, конвертация 4,5 с, `scripts/make_curated.py`) даёт 0,14 с, примерно в 73 раз быстрее «до»:

```
Исходный объект s3://hh-lake/raw/it_vacancies_2024.csv: 847,619,384 байт
```

```
s3://hh-lake/curated/vacancies.parquet: 8,170,694 байт (1.0 % исходного), записан за 4.5 с
s3://hh-lake/segments/go/it_vacancies_go.csv: 10,699,361 байт (1.3 % исходного), записан за 3.6 с
Строк: Parquet 229,388, сегмент Go 2,751
```

Полный вывод замеров — в `logs/08_benchmark.txt`, по нему построены таблицы этого раздела.

9. Выводы

1. RustFS в Docker Compose (4 папки-диска, EC 2+2) хранит в бакете hh-lake датасет и результаты обоих способов для навыков, пар и троек; целостность подтверждена SHA-256.
2. Свой MapReduce читает объект из RustFS сплитами, переживает падение воркера и считает частоты так же, как DuckDB: контрольные числа, все частоты и медианы зарплат совпали. Рост числа пар и троек объясняется числом сочетаний навыков внутри вакансии.
3. Зарплату поднимают ClickHouse, gRPC и Kafka, добавленные к базовым навыкам, и понижают MySQL, REST, Python и Git в связке с SQL, Linux и PostgreSQL: так проявляется разница между вакансиями на высоконагруженные сервисы и младшими смешанными вакансиями.
4. Москва и Петербург близки и по навыкам, и по зарплатам; Екатеринбург младше по опыту, чаще удалённый, с другим набором навыков и более низкой медианой.
5. После оптимизации время обоих способов упирается в чтение из хранилища, и дальше помогает только раскладка данных: Parquet или выделенный сегмент.